

実用メタプログラミング

Practical **PHP** meta programming



PHPカンファレンス沖縄2019
2019-10-12 #phpconokinawa

！ お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE
- GitHub/Packagistでは id: zonuexe
- ピクシブ株式会社 pixiv事業本部
- Emacs Lisper, PHPer
 - Emacs PHP Modeのメンテナ引き継ぎました
 - 好きなリスプはEmacs Lispです
- Qiitaに記事を書いたり変なコメントしてるよ

pixiv



お絵描きがもっと楽しくなる場所を創る

pixiv 2018年度新卒
エントリー受付中



Friends of Emacs-PHP development

Join us!

<?php

- Repositories 25
- People 5
- Teams 1
- Projects 0
- Settings

Find a repository... Type: All Language: All

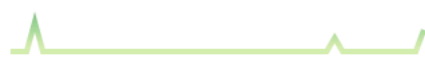
Customize pinned repositories New

php-runtime.el

PHP-Emacs bridge, call PHP function from Emacs

- php
- emacs
- melpa

Emacs Lisp 2 1 GPL-3.0 Updated 2 days ago



phpactor.el

Interface to Phpactor (an intelligent code-completion and refactoring tool for PHP)

Emacs Lisp 8 4 1 issue needs help Updated 3 days ago



php-mode

A PHP mode for GNU Emacs



Top languages

Emacs Lisp PHP

Most used topics Manage

emacs melpa php

major-mode

People 5 >



Invite someone



Search or jump to...



Pull requests

Issues

Marketplace

Explore



emacs-jp

Japan

<http://emacs-jp.github.com/>

Repositories 18

People 37

Teams 8

Projects 0

Settings

Find a repository...

Type: All

Language: All

Customize pinned repositories

New

reading-init.el

init.el読書会のページ

html

dotfiles

emacs

emacs-lisp

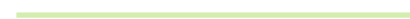
Ruby ★ 6 1 Updated 6 days ago



helm-c-yasnippet

Helm source for yasnippet

Emacs Lisp ★ 14 5 Updated on 21 Mar



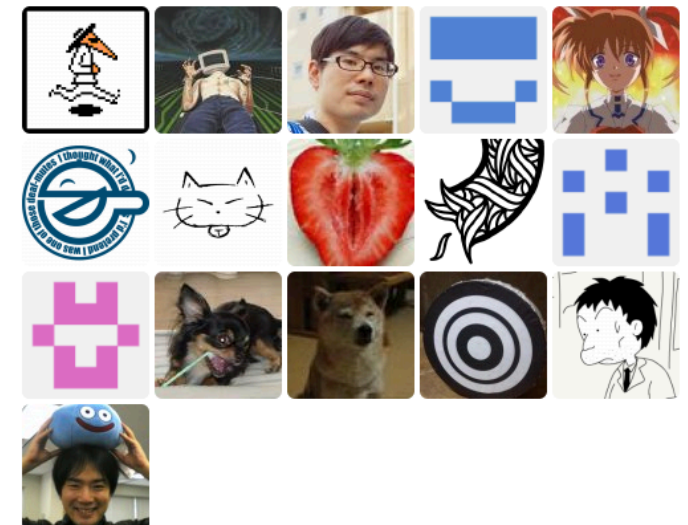
replace-colorthemes

Top languages

Emacs Lisp Ruby TeX

People

37



さて

メタプログラミング とは何か

そもそも

メタとは



ウィキペディア
フリー百科事典

メインページ
コミュニティ・ポータル
最近の出来事
新しいページ
最近の更新
おまかせ表示
練習用ページ
アップロード (ウィキメ
ディア・コモンズ)

ヘルプ

ヘルプ

ログインしていません トーク 投稿記録 アカウント作成 ログイン

ページ

ノート

閲覧

編集

履歴表示

Wikipedia内を検索



メタ

出典: フリー百科事典『ウィキペディア (Wikipedia) 』

ウィキペディアについての議論を行う「メタウィキ」については[Wikipedia:メタウィキメ
ディア](#)をご覧ください。

「メタ」のその他の用法については「[メタ \(曖昧さ回避\)](#)」をご覧ください。

メタ (meta-) とは、「高次なー」「超ー」「一間の」「ーを含んだ」「ーを入れた」「ーの後ろの」等の意味の[接頭語](#)。[ギリシア語](#)から。

例えば物質界の現象を超越した世界を取り扱う学問を「形而上学」というが、これは[英語](#)で metaphysics であり、physics ([物理学](#)) の後の学問として存在する（詳しくは「[形而上学](#)」の項を参照のこと）。

出典: フリー百科事典『ウィキペディア (Wikipedia) 』

<https://ja.wikipedia.org/wiki/メタ>

最終更新 [2019年1月10日 10:01 版](#)

狭義のメタ [編集]

ある対象を記述したものがあり、さらにそれを対象として記述するものを、メタな○○、あるいは単にメタ○○と呼ぶ。

- 言語を記述する言語 - メタ言語
- 文法を記述する文法 - メタ文法
- 理論を解釈するための理論 - メタ理論(理論に対する理論の意味)
- 自己の認知を認知すること - メタ認知

例えば、[WWW](#)上の文章は主に[HTML](#)によって記述されているが、この場合、元の文章に対してHTMLがメタな言語であるといえる。また、HTMLそのものは[英語](#)によって仕様が規定されている。この場合、HTMLにとって英語がメタな言語であるといえる。

出典: フリー百科事典『ウィキペディア (Wikipedia)』

<https://ja.wikipedia.org/wiki/メタ>

最終更新 2019年1月10日 10:01 版

広義のメタ [編集]

広義に、何かを取り込んだ何か、何かについての何か、といったものがメタと呼ばれる場合がある。たとえば、Wikipediaの記事**一覧の一覧**などはメタ的な記事の一つだろう。**情報理論**では、**情報**に付加されるそれ自身に関する情報が**メタデータ**（メタ情報）と呼ばれる。

また、**小説**をテーマにした小説や、**映画**をテーマにした映画などがメタな小説（あるいはメタ小説）、メタな映画などという風に言われることがある。この「小説内小説」「映画内映画」と言った入れ子構造は「小説とは何か、映画とは何か」という、それ自体についての**自意識**を如実に表している（フィクションがフィクションであることを表している）ことが多く、それぞれ研究の対象となる（**メタフィクション**を参照）。

この**メタフィクション**を元にした**俗語**に「メタい/メタ発言」などがあるが、これは「とある作中人物が自らをその作品の登場人物の一人であることを認識している様、また、それを示唆する発言」を意味する。

出典: フリー百科事典『ウィキペディア (Wikipedia) 』

<https://ja.wikipedia.org/wiki/メタ>

最終更新 2019年1月10日 10:01 版

メタプログラミング とは何か

プログラム自身に
言及するような
プログラミング

プログラムを
プログラミングする

どうということ？

たとえば実行時に
関数を定義する

たとえば実行時に
実行する関数を決める

たとえばプロパティや
メソッドのアクセス
権限を制御する

たとえば実装コード
から自動でテスト
コードを生成する

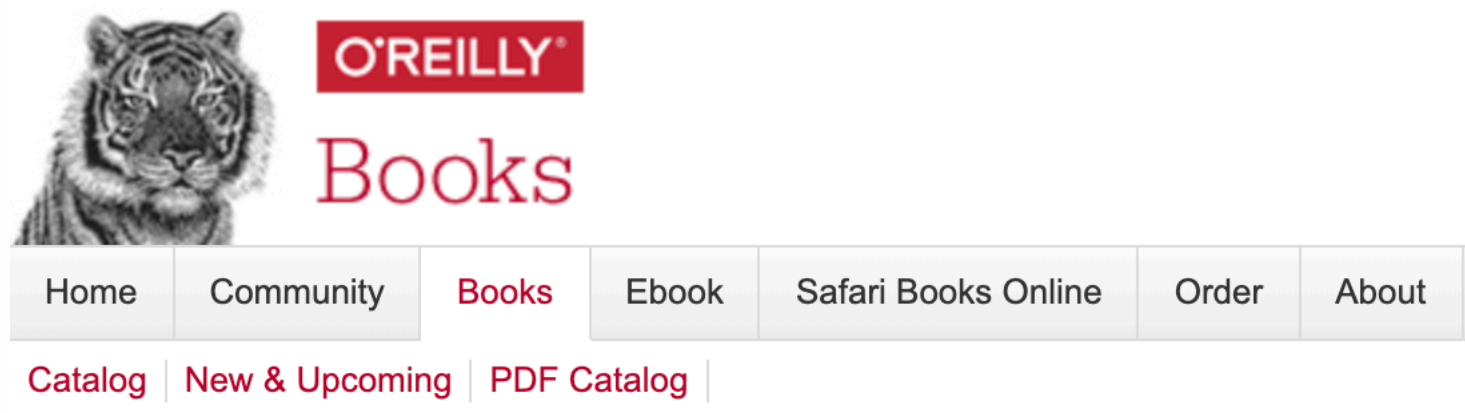
厳密にプログラミング
とメタプログラミング
を分けるものはないが

フレームワークは大なり
小なりメタプログラミング
を利用している
(ことが多い)

PHPの可能性を
広げるのが
メタプログラミング

メタプログラミング
できることで
有名な言語

Ruby

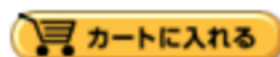


メタプログラミングRuby 第2版

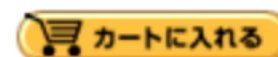


Paolo Perrotta 著、角 征典 訳
2015年10月 発行
292ページ
ISBN978-4-87311-743-0
原書: [Metaprogramming Ruby 2](#)
フォーマット ☐ 本 ☒ PDF ☐ EPUB ☐ mobi

オライリー・ジャパンで書籍を購入:
定価3,300円



Ebook Storeで電子版を購入:
価格2,640円



[ツイート](#) [B!](#) [ええやん!](#) 35

出典: O'Reilly Japan - メタプログラミングRuby 第2版
<https://www.oreilly.co.jp/books/9784873117430/>
Paolo Perrotta 著 / 角征典 訳 / 2015年10月 発行

Lisp

メタプログラミング 37 users

出典: フリー百科事典『ウィキペディア（Wikipedia）』

メタプログラミング (metaprogramming) とは **プログラミング** 技法の一種で、ロジックを直接コーディングするのではなく、あるパターンをもったロジックを生成する高位ロジックによってプログラミングを行う方法、またその高位ロジックを定義する方法のこと。主に対象言語に埋め込まれた **マクロ言語** によって行われる。

概要 [[ソースを編集](#)]

代表的なメタプログラミングの例は **LISP** のマクロである。LISP ではデータ、コードが全て **S式** で表現されるが、マクロにより S 式が言語処理系に解釈される前に別な S 式へと変換することができる。これにより例えば、

```
(defstruct point (x 0) (y 0))
```

出典: フリー百科事典『ウィキペディア（Wikipedia）』

<https://ja.wikipedia.org/wiki/メタプログラミング>

最終更新 **2017年12月6日 23:08 版**

PHPではできない
ことの方が多い

オーブンクラスがない
プリプロセスもない

PHPならではの
戦いかたがある！

関数

FizzBuzzで
考えてみましょう

どうしてプログラマに・・・プログラムが書けないのか？

Jeff Atwood / 青木靖 訳

2007年2月26日

993 users 

[レジナルド・ブレイスウェイトが書いていること](#)を読んだとき、私はそんなわけないだろうと思っていた。

私と同様、この著者は、プログラミングの仕事への応募者[200人中199人](#)はコードがまったく書けないということで苦労している。繰り返すが、彼らはどんなコードも書けないのだ。

彼が引用している著者というのはイムランのことで、彼は[単純なプログラムも書けないプログラマをたくさん追い払っている](#)ということだ。

出典: どうしてプログラマに・・・プログラムが書けないのか？

http://www.aoky.net/articles/jeff_atwood/why_cant_programmers_program.htm

Jeff Atwood著 / 青木靖 訳

1から100までの数をプリントするプログラムを書け。ただし3の倍数のときは数の代わりに「Fizz」と、5の倍数のときは「Buzz」とプリントし、3と5両方の倍数の場合には「FizzBuzz」とプリントすること。

出典: どうしてプログラマに・・・プログラムが書けないのか?

http://www.aoky.net/articles/jeff_atwood/why_cant_programmers_program.htm

Jeff Atwood著 / 青木靖 訳

| FizzBuzz模範回答

```
<?php
```

```
foreach (range(1, 100) as $n) {  
    $s = '';  
    if ($n % 3 === 0) $s .= 'Fizz';  
    if ($n % 5 === 0) $s .= 'Buzz';  
    echo ($s === '' ) ? $n : $s, "\n";  
}
```


| FizzBuzz模範回答 関数分割

```
<?php
function to_fizzbuzz(int $n) {
    $s = '';
    if ($n % 3 === 0) $s .= 'Fizz';
    if ($n % 5 === 0) $s .= 'Buzz';
    return ($s === '') ? $n : $s;
}

foreach (range(1, 100) as $n) {
    echo to_fizzbuzz($n), "\n";
}
```

関数に分けるとテストが書ける

```
<?php
```

```
assert(to_fizzbuzz(1) === "1");  
assert(to_fizzbuzz(2) === "2");  
assert(to_fizzbuzz(3) === "Fizz");  
assert(to_fizzbuzz(4) === "4");  
assert(to_fizzbuzz(5) === "Buzz");
```

無茶振りをしましょう

関数に分けたい
でも引数使いたくない

| 愚直に書くようになる

```
<?php
```

```
assert(fizzbuzz1() === "1");  
assert(fizzbuzz2() === "2");  
assert(fizzbuzz3() === "Fizz");  
assert(fizzbuzz4() === "4");  
assert(fizzbuzz5() === "Buzz");
```

```
function fizzbuzz1() { return "1"; }  
function fizzbuzz2() { return "2"; }  
function fizzbuzz3() { return "Fizz"; }  
function fizzbuzz4() { return "4"; }  
function fizzbuzz5() { return "Buzz"; }
```

手書きで
100個書くの!?

| こういうことはできない

<?php // このコードはsyntax errorになります

```
foreach (range(1, 100) as $n) {  
    $s = '';  
    if ($n % 3 === 0) $s .= 'Fizz';  
    if ($n % 5 === 0) $s .= 'Buzz';  
  
    // 変数を使った関数定義はできない  
  
    function fizzbuzz$n() {  
        return ($s === '') ? $n : $s, "\n";  
    }  
}
```


| [非推奨] evalを使った無茶

```
<?php
```

```
foreach (range(1, 100) as $n) {  
    $s = '';  
    if ($n % 3 === 0) $s .= 'Fizz';  
    if ($n % 5 === 0) $s .= 'Buzz';  
    $s = ($s === '') ? $n : $s;  
    // evalを使えばいちおうできる  
    eval("function fizzbuzz{$n} () {  
        return '{$s}';  
    }");  
}
```


evalはなんでも
できるがリスクも大きい
最後の手段

(create_function()関数も
あったがPHP7.2で非推奨)

無名関数(クロージャ)
を使いましょう

| クロージャを使えばできる

```
<?php
$fizzbuzz = [];
foreach (range(1, 100) as $n) {
    $s = '';
    if ($n % 3 === 0) $s .= 'Fizz';
    if ($n % 5 === 0) $s .= 'Buzz';
    $s = ($s === '') ? (string)$n : $s;

    $fizzbuzz[$n] = function () use ($s) {
        return $s;
    };
}
```

クロージャの特徴

- 関数定義文ではなく関数式
- 使い捨て関数を簡単に作れる
- 関数式はuseで変数捕捉できる
- Closureクラスのオブジェクトで、変数や引数として扱える

| これで呼び出せるようになった

```
<?php
```

```
assert( $fizzbuzz[ 1 ] ( ) === "1" );  
assert( $fizzbuzz[ 2 ] ( ) === "2" );  
assert( $fizzbuzz[ 3 ] ( ) === "Fizz" );  
assert( $fizzbuzz[ 4 ] ( ) === "4" );  
assert( $fizzbuzz[ 5 ] ( ) === "Buzz" );
```

クロージヤを使った
コード

| クロージャ + array_map

```
<?php
```

```
$x2 = function ($n){ return $n * 2; };
```

```
$a = range(1, 10);
```

```
var_dump(array_map($x2, $a));
```

```
// [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```


クロージャをうまく使って
「関数を作る関数」も
作れる

| クロージャを使った部分適用

```
<?php
```

```
$times = function ($n) {  
    return function ($m) use ($n) {  
        return $n * $m;  
    };  
};
```

```
$a = range(1, 10);  
echo json_encode(array_map($times(3), $a));  
// [3,6,9,12,15,18,21,24,27,30]
```

マジックメソッド

オブジェクトの
挙動を制御する
特殊なメソッド

プロパティアクセス
を乗っ取ろう

| アクセス不能プロパティを乗っ取る

```
<?php
class A {
    private $data = [];

    public function __get( $name ) {
        $this->data[ $name ];
    }

    public function __set( $name, $value ) {
        $this->data[ $name ] = $value;
    }
}
```

__isset() と __unset() も定義すべき

アクセス不能
=privateなども含む

この仕様を悪用して
こんなこともできる

| 外部から読み込みだけできる

```
<?php
trait PrivateRead {
    public function __get($name) {
        $this->$name;
    }
}

/**
 * @property-read int $id
 * @property-read int $name
 */
class A {
    use PrivateRead;
    private $id;
    private $name;
}
```


| 未定義プロパティに書き込み禁止

```
<?php
```

```
trait ReadOnly {  
    public function __set($name, $value) {  
        throw new OutOfRangeException();  
    }  
}
```

```
class B {  
    use ReadOnly;  
}
```

メソッドアクセスも
乗っ取ろう

オーバーロード
(ただしPHP用語)

Change language: Japanese[Edit](#)[Report a Bug](#)

オーバーロード

PHP におけるオーバーロード機能は、プロパティやメソッドを動的に「作成する」ための手法です。これらの動的エンティティは、マジックメソッドを用いて処理されます。マジックメソッドは、クラス内でさまざまなアクションに対して用意することができます。

オーバーロードメソッドが起動するのは、宣言されていないプロパティやメソッドを操作しようとしたときです。また、現在のスコープからは [アクセス不能な](#) プロパティやメソッドを操作しようとしたときにも起動します。このセクションでは、これらの (宣言されていない、あるいは現在のスコープからはアクセス不能な) プロパティやメソッドのことを「アクセス不能プロパティ」および「アクセス不能メソッド」と表記することにします。

オーバーロードメソッドは、すべて *public* で定義しなければなりません。

出典: PHP: オーバーロード - Manual

<https://www.php.net/manual/ja/language.oop5.overloading.php>

Copyright © 2001-2019 The PHP Group

一般的なプログラミング
用語「オーバーロード」
とは全然別物なので注意

| 未定義のメソッド呼び出しを捕捉

```
<?php
```

```
class C {  
    public function __call($name, $args)  
    {  
        echo __CLASS__."::{$name}()が呼ばれた\n";  
    }  
}
```

| Proxyパターンを簡単に実現

```
<?php
```

```
/**
```

```
 * @mixin Foo
```

```
 */
```

```
class D {
```

```
    private $foo;
```

```
    public function (Foo $class) {
```

```
        $this->foo = $class;
```

```
    }
```

```
    public function __call($name, $args)
```

```
    {
```

```
        return $this->foo->$name(...$args);
```

```
    }
```

```
}
```


リフレクシヨン

実行時に情報を取得
(一部上書き)できるAPI

関数・クラス・オブジェクト・
メソッド・プロパティ・パラメー
タ(引数)・返値の型・クラス定数
など

<https://www.php.net/manual/ja/book.reflection.php>

PHPUnitの
dataProvider
(パラメタライズテスト)
とも組み合わせられる

| 未定義のメソッド呼び出しを捕捉

```
<?php
```

```
class C {  
    public function __call($name, $args)  
    {  
        echo __CLASS__."::{$name}()が呼ばれた\n";  
    }  
}
```

まとめ

メタプログラミングの技術を使
うと
「コードに書いてある通りで
はない」動きをさせられる

代償としてPhpStorm
などの支援を受けにく
くなることがある

大いなる力には
大いなる責任が伴う